

EXAMEN DE PROGRAMACIÓN – 1º GRADO SUPERIOR

Examen 6 · Recursividad y algoritmos recursivos

Recursión: concepto, caso base y caso recursivo

La pila de llamadas y el desbordamiento de pila (StackOverflowError)

Recursión directa, indirecta y recursión de cola

Algoritmos recursivos clásicos: factorial, Fibonacci, MCD de Euclides

Búsqueda binaria recursiva y Torres de Hanoi

Asignatura	Programación (PRG)
Curso	1º Grado Superior — Desarrollo de Aplicaciones Web (DAW)
Duración	2 horas
Puntuación total	16 puntos
Convocatoria	Evaluación 2
Valoración	Teoría y práctica de algoritmos recursivos

Parte 1: Opción múltiple y Verdadero/Falso

4 puntos — 0,5 cada pregunta

1. Dado el siguiente método recursivo, ¿qué devuelve `fact(4)`?

```
static int fact(int n) {  
    if (n <= 1) return 1;  
    return n * fact(n - 1);  
}
```

- a) 12
- b) 24
- c) 4
- d) 120

RESPUESTA / SOLUCIÓN

b) 24 — $\text{fact}(4) = 4 \cdot \text{fact}(3) = 4 \cdot 3 \cdot \text{fact}(2) = 4 \cdot 3 \cdot 2 \cdot \text{fact}(1) = 4 \cdot 3 \cdot 2 \cdot 1 = 24$. El caso base ($n \leq 1$) detiene la recursión.

-
2. Con el método Fibonacci clásico (`fib(0)=0`, `fib(1)=1`, `fib(n)=fib(n-1)+fib(n-2)`), ¿cuánto vale `fib(6)`?

- a) 5
- b) 6
- c) 8
- d) 13

RESPUESTA / SOLUCIÓN

c) 8 — La sucesión es 0, 1, 1, 2, 3, 5, 8; por tanto `fib(6) = 8`.

-
3. Toda función recursiva debe tener al menos un caso base; sin él se produce un desbordamiento de pila (`StackOverflowError`).

RESPUESTA / SOLUCIÓN

Verdadero — Sin caso base la función se llama a sí misma indefinidamente y agota la memoria de la pila de llamadas, lanzando `StackOverflowError`.

4. En las Torres de Hanoi, ¿cuántos movimientos son necesarios como mínimo para mover una torre de 3 discos?

- a) 3

- b) 5
- c) 7
- d) 9

RESPUESTA / SOLUCIÓN

c) 7 — El número mínimo de movimientos es $2^n - 1$; para $n = 3$ discos: $2^3 - 1 = 7$.

5. Usando el algoritmo de Euclides recursivo, ¿qué devuelve `mcd(48, 18)`?

```
static int mcd(int a, int b) {  
    if (b == 0) return a;  
    return mcd(b, a % b);  
}
```

- a) 3
- b) 6
- c) 9
- d) 12

RESPUESTA / SOLUCIÓN

b) 6 — $\text{mcd}(48, 18) \rightarrow \text{mcd}(18, 12) \rightarrow \text{mcd}(12, 6) \rightarrow \text{mcd}(6, 0) = 6$.

6. Con la búsqueda binaria recursiva sobre el array ordenado `{2, 5, 8, 12, 19, 25}`, ¿qué índice devuelve `buscar(19)`?

- a) 3
- b) 4
- c) 5
- d) -1

RESPUESTA / SOLUCIÓN

b) 4 — El valor 19 está en la posición 4 del array (los índices empiezan en 0).

7. En la recursión de cola (tail recursion), la llamada recursiva es la última operación que ejecuta el método, lo que permite en algunos lenguajes optimizar la llamada reutilizando el marco de pila.

RESPUESTA / SOLUCIÓN

Verdadero — En la recursión de cola no queda trabajo pendiente después de la llamada recursiva; algunos compiladores/VMs la optimizan (aunque Java no la optimiza de forma general).

8. ¿Qué devuelve sumaDig(345) con el siguiente método?

```
static int sumaDig(int n) {  
    if (n < 10) return n;  
    return n % 10 + sumaDig(n / 10);  
}
```

- a) 12
- b) 15
- c) 8
- d) 345

RESPUESTA / SOLUCIÓN

a) 12 — $5 + \text{sumaDig}(34) = 5 + 4 + \text{sumaDig}(3) = 5 + 4 + 3 = 12$.

Parte 2: Análisis y depuración

3 puntos — 1,5 cada ejercicio

1. El siguiente programa tiene 2 errores. Encuéntralos, explica qué ocurre al ejecutarlo y escribe la versión corregida. (1,5 pts)

```
public class PotenciaBug {  
    static int potencia(int base, int exp) {  
        return base * potencia(base, exp - 1);  
    }  
    public static void main(String[] args) {  
        System.out.println(potencia(2, 3));  
    }  
}
```

RESPUESTA / SOLUCIÓN

1. Falta el caso base: la función se llama a sí misma sin condición de parada, por lo que la recursión es infinita y el programa termina con `StackOverflowError`.
2. Tampoco se contempla el caso `exp == 0` (debería devolver 1) ni los exponentes negativos, que nunca llegarían a 0.

Versión corregida:

```
static int potencia(int base, int exp) {  
    if (exp == 0) return 1;           // caso base  
    return base * potencia(base, exp - 1); // caso recursivo  
}
```

-
2. Analiza el siguiente método y realiza la traza de la ejecución de `fib(5)`. Indica el valor devuelto y el número total de llamadas a `fib` que se realizan. (1,5 pts)

```
static int fib(int n) {  
    if (n < 2) return n;  
    return fib(n - 1) + fib(n - 2);  
}
```

RESPUESTA / SOLUCIÓN

Valores: `fib(0)=0`, `fib(1)=1`, `fib(2)=1`, `fib(3)=2`, `fib(4)=3`, `fib(5)=5`.

Árbol de llamadas de `fib(5)`: `fib(5)` llama a `fib(4)` y `fib(3)`; `fib(4)` llama a `fib(3)` y `fib(2)`; `fib(3)` llama a `fib(2)` y `fib(1)`; `fib(2)` llama a `fib(1)` y `fib(0)`.

Total de llamadas: 15 (1 de `fib(5)` + 2 del nivel 2 + 4 del nivel 3 + 8 del nivel 4).

Resultado: `fib(5) = 5`.

Parte 3: Ejercicios de programación

8 puntos — 2 cada ejercicio

1. Implementa un método recursivo `potencia(int base, int exp)` que calcule $base^{exp}$ para exponentes mayores o iguales que 0. Escribe también un main que pruebe 2^{10} , 3^0 y 5^3 . (2 pts)

RESPUESTA / SOLUCIÓN

```
public class Potencia {
    static long potencia(int base, int exp) {
        if (exp == 0) return 1;           // caso base
        return base * potencia(base, exp - 1); // caso recursivo
    }
    public static void main(String[] args) {
        System.out.println("2^10 = " + potencia(2, 10)); // 1024
        System.out.println("3^0 = " + potencia(3, 0)); // 1
        System.out.println("5^3 = " + potencia(5, 3)); // 125
    }
}
```

Nota: para exponentes negativos haría falta trabajar con double ($base^{-n} = 1 / base^n$).

-
2. Escribe una función recursiva que devuelva una cadena invertida y un main que la pruebe con la cadena «Hola mundo». (2 pts)

RESPUESTA / SOLUCIÓN

```
public class InvertirCadena {
    static String invertir(String s) {
        if (s.length() <= 1) return s;           // caso base
        return invertir(s.substring(1)) + s.charAt(0);
    }
    public static void main(String[] args) {
        String texto = "Hola mundo";
        System.out.println(invertir(texto)); // "odnum aloH"
    }
}
```

Explicación: `invertir("Hola") = invertir("ola") + 'H' = (invertir("la") + 'o') + 'H' = ... = "aloH"`.

-
3. Implementa la solución recursiva de las Torres de Hanoi para n discos: el método debe imprimir cada movimiento («Mover disco X de A a C») y el main debe ejecutarlo con 3 discos mostrando además el número total de movimientos. (2

ptos)

RESPUESTA / SOLUCIÓN

```
public class Hanoi {
    static int movimientos = 0;

    static void hanoi(int n, char origen, char auxiliar, char destino) {
        if (n == 1) {
            System.out.println("Mover disco 1 de " + origen + " a " + destino);
            movimientos++;
        } else {
            hanoi(n - 1, origen, destino, auxiliar);
            System.out.println("Mover disco " + n + " de " + origen + " a " + destino);
            movimientos++;
            hanoi(n - 1, auxiliar, origen, destino);
        }
    }
}

public static void main(String[] args) {
    hanoi(3, 'A', 'B', 'C');
    System.out.println("Total de movimientos: " + movimientos); // 7
}
```

El número mínimo de movimientos es $2^n - 1$; para 3 discos se imprimen 7 movimientos.

-
4. Implementa la búsqueda binaria de forma recursiva sobre un array ordenado de enteros. Escribe un main que busque el 19 y el 7 en {2, 5, 8, 12, 19, 25} mostrando el índice encontrado o «No encontrado». (2 pts)

RESPUESTA / SOLUCIÓN

```
public class BusquedaBinaria {
    static int buscar(int[] a, int ini, int fin, int x) {
        if (ini > fin) return -1;           // caso base: no encontrado
        int medio = (ini + fin) / 2;
        if (a[medio] == x) return medio;    // caso base: encontrado
        if (x < a[medio]) return buscar(a, ini, medio - 1, x);
        return buscar(a, medio + 1, fin, x);
    }
    public static void main(String[] args) {
        int[] a = {2, 5, 8, 12, 19, 25};
        int pos = buscar(a, 0, a.length - 1, 19);
        System.out.println(pos >= 0 ? "Índice: " + pos : "No encontrado"); // 4
        pos = buscar(a, 0, a.length - 1, 7);
        System.out.println(pos >= 0 ? "Índice: " + pos : "No encontrado"); // No encontrado
    }
}
```

Cada llamada divide el intervalo a la mitad, por lo que la complejidad es $O(\log n)$.

Parte 4: Pregunta teórica

1 punto

1. Explica qué es la recursión, qué papel juegan el caso base y la pila de llamadas, y en qué situaciones conviene usar recursión en lugar de iteración. ¿Qué riesgo tiene un uso incorrecto?

RESPUESTA / SOLUCIÓN

La recursión es una técnica en la que un método se llama a sí mismo para resolver un problema dividiéndolo en versiones más pequeñas del mismo problema.

El caso base es la condición que detiene las llamadas (p. ej., $n \leq 1$ en el factorial); sin él la recursión es infinita. Cada llamada pendiente ocupa un marco en la pila de llamadas (stack), que guarda sus parámetros y variables locales; al llegar al caso base, los marcos se desapilan devolviendo resultados.

Conviene usar recursión cuando el problema se define de forma natural recursiva (árboles, directorios, Hanoi, divide y vencerás) y cuando la versión iterativa sería mucho más compleja. La iteración es preferible cuando la profundidad puede ser grande, porque la recursión consume memoria de pila y un uso incorrecto (sin caso base o con demasiada profundidad) lanza `StackOverflowError`.



Plantilla de respuestas

(Páginas en blanco para desarrollar las soluciones)